Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

Implementación de un modelo criptográfico híbrido poscuántico para dispositivos IoT de bajo costo

Diagrama de bloques, descripción de los bloques y diagramas de flujo

1) Diagrama de bloques general de la propuesta

La propuesta se implementa en un banco de pruebas formado por dos dispositivos de bajo costo que se comunican únicamente por un bus I2C dedicado: un nodo sensor basado en el microcontrolador ESP32, que actúa como cliente TLS 1.3, y una unidad de borde basada en una Raspberry Pi 4, que actúa como servidor TLS 1.3 y como maestro del bus. Sobre ese enlace se establecen sesiones TLS con dos tipos de intercambio de claves: X25519, como línea base clásica, y X25519MLKEM768, como intercambio híbrido poscuántico. Un sensor de corriente y voltaje INA219 mide la potencia del sistema durante cada corrida, y los datos obtenidos se procesan para comparar ambos intercambios. La Figura 1 muestra la organización general de los bloques.

En cada corrida, el orquestador de la Raspberry Pi envía al ESP32 la orden de inicio con el tipo de intercambio y el tamaño del mensaje. El ESP32 abre la sesión TLS, envía la telemetría cifrada y un bloque de 4 KiB, y al final reporta sus tiempos y su memoria. Todos los mensajes, incluido el handshake, viajan en tramas por el bus I2C, mientras el INA219 registra la potencia en paralelo por otro bus. Al terminar la campaña, el análisis combina las métricas de cada corrida con el registro de potencia para obtener la energía de cada handshake. La Tabla 1 resume los componentes del banco de pruebas.

Tabla 1

Componentes del banco de pruebas

Componente	Elemento	Características
Nodo sensor	ESP32 (ESP-IDF 5.5.5)	Cliente TLS 1.3 con wolfSSL 5.9.4; esclavo I2C en 0x28
Unidad de borde	Raspberry Pi 4	Servidor TLS 1.3 con OpenSSL 3.5.7; maestro del bus 3
Enlace	Bus I2C dedicado	100 o 400 kbit/s; tramas de 128 bytes; línea READY
Intercambio de claves	X25519 / X25519MLKEM768	Línea base clásica / híbrido poscuántico
Medición	Sensor INA219	Voltaje, corriente y potencia en la alimentación de 5 V; bus 1
Seguridad	Analizador lógico	Captura independiente del tráfico del bus

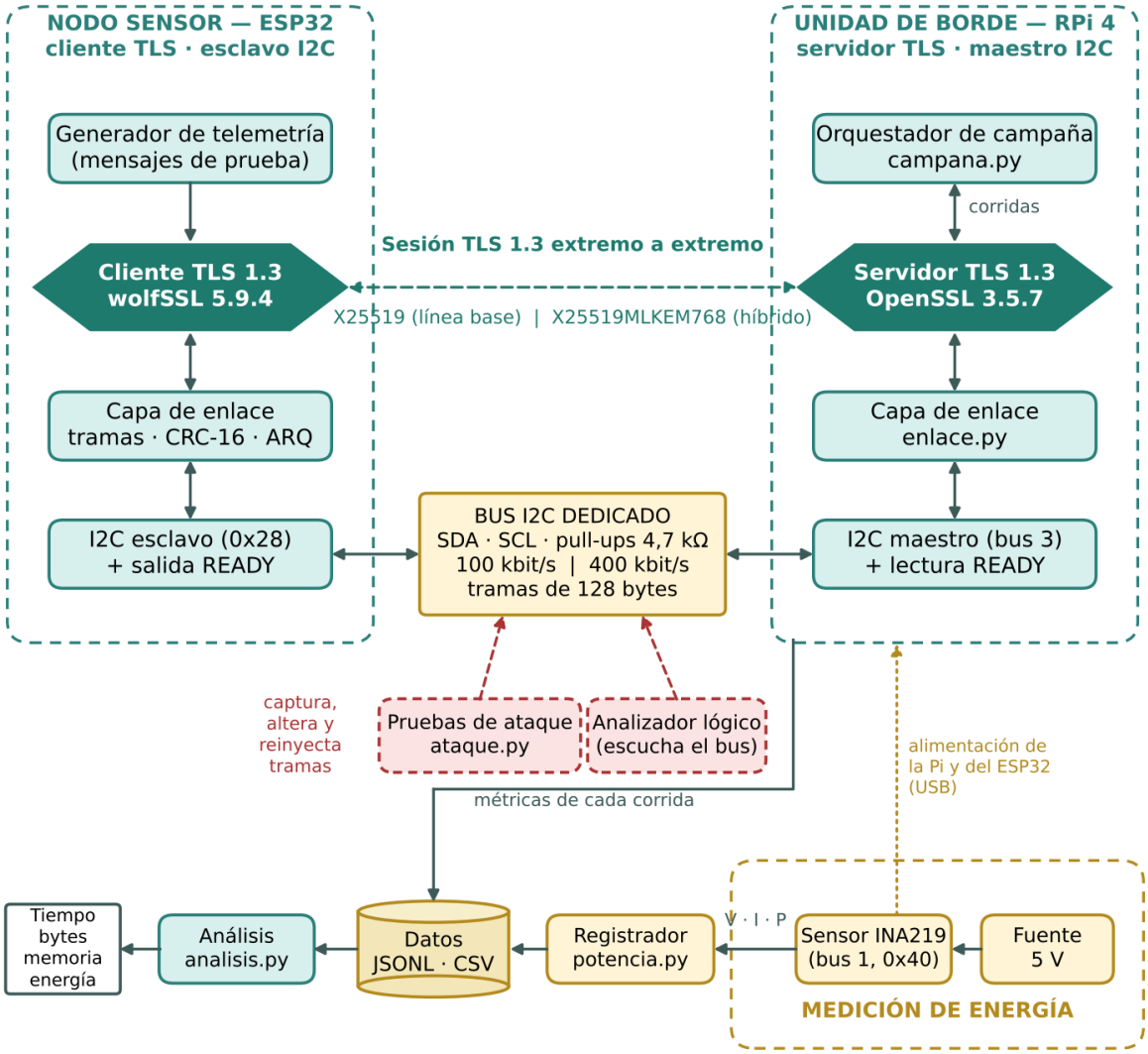
Nota. Elaboración propia.

Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

Figura 1

Diagrama de bloques general del banco de pruebas



Nota. Elaboración propia. Las flechas continuas indican flujo de datos; la flecha discontinua verde, la sesión TLS lógica entre ambos extremos; las cajas rojas discontinuas, los elementos usados solo en las pruebas de seguridad; y la línea punteada ámbar, la alimentación medida por el INA219.

2) Descripción de los bloques

Los bloques se agrupan en cinco partes: el nodo sensor, el bus I2C, la unidad de borde, la medición de energía con el análisis de datos y las pruebas de seguridad. A continuación se describe la función de cada uno.

2.1 Nodo sensor (ESP32)

- **Generador de telemetría.** Produce los mensajes de prueba que simulan las lecturas de un sensor, con tamaños de 32, 128 y 512 bytes, y el bloque de 4 KiB usado para medir el rendimiento útil del canal cifrado.

- **Cliente TLS 1.3 (wolfSSL 5.9.4).** Inicia el handshake con la Raspberry Pi ofreciendo el grupo X25519 o X25519MLKEM768, verifica el certificado del servidor con la autoridad de certificación del proyecto y cifra la telemetría. Además, mide la duración del handshake y la memoria máxima que usa la biblioteca, mediante un contador propio de reservas de memoria. Como el ESP32 no dispone de reloj de tiempo real, el reloj se fija a la fecha de compilación para que la validación del certificado sea posible.
- **Capa de enlace (tramas, CRC-16 y ARQ).** TLS necesita un transporte confiable y ordenado, que I2C no ofrece. Este bloque divide el flujo TLS en tramas de 128 bytes con la estructura MAGIC | TIPO | SEQ | ACK | LEN | carga útil | CRC-16, detecta errores con el código CRC-16/CCITT y retransmite las tramas no confirmadas mediante un esquema de parada y espera.
- **I2C esclavo (0x28) y salida READY.** El ESP32 responde en la dirección 0x28 del bus. Cuando tiene preparada la siguiente respuesta, activa la línea READY (GPIO4 del ESP32 hacia GPIO17 de la Raspberry Pi), lo que evita que el maestro lea datos incompletos.

2.2 Bus I2C dedicado

Es el único enlace entre ambos dispositivos. Usa las líneas SDA y SCL del bus 3 de la Raspberry Pi (GPIO4 y GPIO5) con resistencias de pull-up de 4,7 kΩ a 3,3 V, y se evalúa en modo estándar (100 kbit/s) y en modo rápido (400 kbit/s). Todo el tráfico, incluido el handshake, viaja en tramas de 128 bytes, de los cuales 120 son carga útil.

2.3 Unidad de borde (Raspberry Pi 4)

- **Orquestador de campaña (campana.py).** Genera un plan aleatorio de corridas que combina el tipo de intercambio y el tamaño del mensaje, controla la temperatura del procesador, inicia el registro de potencia y deja ventanas de reposo al inicio y al final para medir la potencia base.
- **Servidor TLS 1.3 (OpenSSL 3.5.7).** Atiende el handshake, identifica el grupo negociado, devuelve el eco cifrado de la telemetría, recibe el bloque de rendimiento y registra los tiempos de cada etapa y los bytes intercambiados (servidor_tls.py).
- **Capa de enlace (enlace.py).** Es la contraparte de la capa de enlace del ESP32 en el lado maestro: arma las tramas, verifica el CRC, gestiona los números de secuencia y las retransmisiones, y lleva los contadores de errores de CRC, retransmisiones y tiempos de espera.
- **I2C maestro (bus 3) y lectura de READY.** Escribe cada trama al ESP32, espera la señal READY con un límite de 200 ms y lee la respuesta. Por cada escritura se realiza siempre una lectura, lo que mantiene sincronizados ambos extremos.

2.4 Medición de energía, datos y análisis

- **Fuente de 5 V.** Alimenta la Raspberry Pi, que a su vez alimenta al ESP32 por USB. Toda la corriente del sistema pasa por el sensor.
- **Sensor INA219 (bus 1, dirección 0x40).** Mide el voltaje y la corriente de alimentación con resolución de 12 bits. Se lee por un bus I2C distinto del bus del experimento, para no interferir con el tráfico medido.
- **Registrador (potencia.py).** Lee el sensor de forma continua en un núcleo del procesador reservado, calcula la potencia y guarda cada muestra con su marca de tiempo en un archivo CSV.
- **Datos (JSONL y CSV).** Por cada corrida se guarda una línea JSON con las métricas del ESP32, del servidor y del enlace; la potencia se guarda en un CSV y la configuración del bloque en un archivo meta.json.
- **Análisis (analisis.py y tamano_muestra.py).** Calcula la potencia de reposo y la energía de cada handshake como la integral de la potencia adicional respecto al reposo, descarta las corridas no válidas, obtiene medias e intervalos de confianza del 95 %, compara el intercambio híbrido con el clásico y genera tablas y figuras. Con los ensayos piloto, tamano_muestra.py determina el número de repeticiones necesario.
- **Métricas de salida.** Tiempo del handshake y de sus etapas, bytes transmitidos por el bus, memoria máxima del ESP32, rendimiento útil y energía por handshake.

2.5 Pruebas de seguridad

- **Pruebas de ataque (ataque.py).** Simulan un atacante con acceso físico al bus: registran las tramas para comprobar que la telemetría no aparece en claro cuando se usa TLS, y alteran o reinyectan tramas recalculando su CRC, de modo que la capa de enlace las acepta y debe ser TLS quien las rechace.
- **Analizador lógico.** Se conecta en paralelo a SDA y SCL y captura el tráfico real del bus, de forma independiente al software del proyecto, como evidencia de lo que puede observar un tercero conectado al bus.

3) Diagramas de flujo de los programas

El proyecto incluye programas para el ESP32, escritos en C sobre ESP-IDF, y para la Raspberry Pi, escritos en Python. La Tabla 2 relaciona cada programa con su función y con la figura que contiene su diagrama de flujo.

Tabla 2

Programas del proyecto y diagramas de flujo correspondientes

Programa	Dispositivo	Función	Figura
main.c, cliente_tls.c	ESP32	Arranque, tareas y cliente TLS	2
enlace.c, enlace.py	ESP32 y Raspberry Pi	Capa de enlace sobre I2C	3
servidor_tls.py	Raspberry Pi	Servidor TLS y ejecución de una corrida	4
campana.py	Raspberry Pi	Orquestación de la campaña de mediciones	5
potencia.py	Raspberry Pi	Registro de potencia con el INA219	6
ataque.py	Raspberry Pi	Pruebas de confidencialidad e integridad	7
analisis.py, tamano_muestra.py	Raspberry Pi	Energía, estadística y tamaño de muestra	8

Nota. Elaboración propia. Los programas están en las carpetas esp32/firmware/main/ y pi/ del repositorio del proyecto.

3.1 Firmware del nodo sensor

Al arrancar, el ESP32 configura la línea READY, la cola de tramas y el periférico I2C en modo esclavo, y crea dos tareas en núcleos distintos: la tarea de enlace, que atiende el bus, y la tarea TLS, que ejecuta el cliente. La tarea TLS fija el reloj, inicia wolfSSL y espera las órdenes de la Raspberry Pi. Por cada orden ejecuta una corrida y, al terminar, envía un reporte con el código de estado, los tiempos y la memoria utilizada; si alguna etapa falla, el reporte lleva un código negativo que identifica la etapa (Figura 2).

3.2 Capa de enlace

La Raspberry Pi, como maestro, inicia todos los intercambios. En cada uno envía una trama de datos, nueva o retransmitida, o una trama de sondeo (POLL) si no tiene nada que enviar; luego espera la señal READY y lee la respuesta. Las tramas con CRC incorrecto se descartan y se cuentan; el ACK recibido libera la trama pendiente, y solo se entregan a TLS las tramas con el número de secuencia esperado, lo que elimina duplicados. En el ESP32, una interrupción recibe la trama, desactiva READY y la pasa a la tarea de enlace, que prepara la respuesta y vuelve a activar READY (Figura 3).

3.3 Servidor TLS y ejecución de una corrida

Cada corrida comienza reiniciando la capa de enlace en ambos extremos y enviando la orden START con el grupo y el tamaño del mensaje. En el grupo de control sin TLS, los mensajes se devuelven en claro; en los demás se realiza el handshake, se registra la duración de cada etapa y los bytes intercambiados, y se identifica el grupo negociado por el tamaño de la respuesta del servidor. Después se devuelve el eco cifrado de la telemetría, se recibe el bloque de rendimiento y se espera el cierre ordenado de la sesión. Cualquier excepción de TLS o tiempo agotado se registra como error de la corrida (Figura 4).

3.4 Orquestador de la campaña de mediciones

El orquestador lee los parámetros del bloque experimental, genera un orden aleatorio de corridas con semilla fija para que pueda reproducirse, y lanza el registrador de potencia antes de iniciar. Luego mide 60 s de reposo, ejecuta las corridas con una pausa de 3 s entre ellas, espera si la temperatura del procesador supera 75 °C, mide otros 60 s de reposo para estimar la deriva y guarda los metadatos del bloque (Figura 5).

3.5 Registrador de potencia

El registrador configura el INA219 con resolución de 12 bits y una muestra por conversión, abre el archivo CSV y lee voltaje y corriente en un ciclo continuo, calculando la potencia y la marca de tiempo de cada muestra. El ciclo termina cuando el orquestador envía la señal de parada o cuando se cumple la duración indicada (Figura 6).

3.6 Pruebas de seguridad

El programa de ataque ejecuta primero dos corridas, sin TLS y con TLS híbrido, registrando todas las tramas, y busca en ellas el texto de la telemetría. Después realiza tres manipulaciones: alterar un bit de un registro cifrado en cada sentido y reinyectar una trama ya enviada, recalculando el CRC para que la capa de enlace no las detecte. La prueba se aprueba si el texto es visible sin TLS, no aparece con TLS y las tres manipulaciones son rechazadas por TLS (Figura 7).

3.7 Análisis de datos y tamaño de muestra

El análisis carga los archivos de cada bloque, calcula la potencia de reposo y su deriva, e integra la potencia adicional durante cada handshake para obtener su energía. Luego marca como válidas las corridas sin errores ni limitación térmica y calcula, por configuración, la media, la desviación y el intervalo de confianza del 95 %. Con los ensayos piloto se aplica el criterio de Jain (1991) para obtener el número de repeticiones, con un mínimo de 30; con la campaña completa se compara el intercambio híbrido con el clásico y se generan las tablas y figuras de resultados (Figura 8).

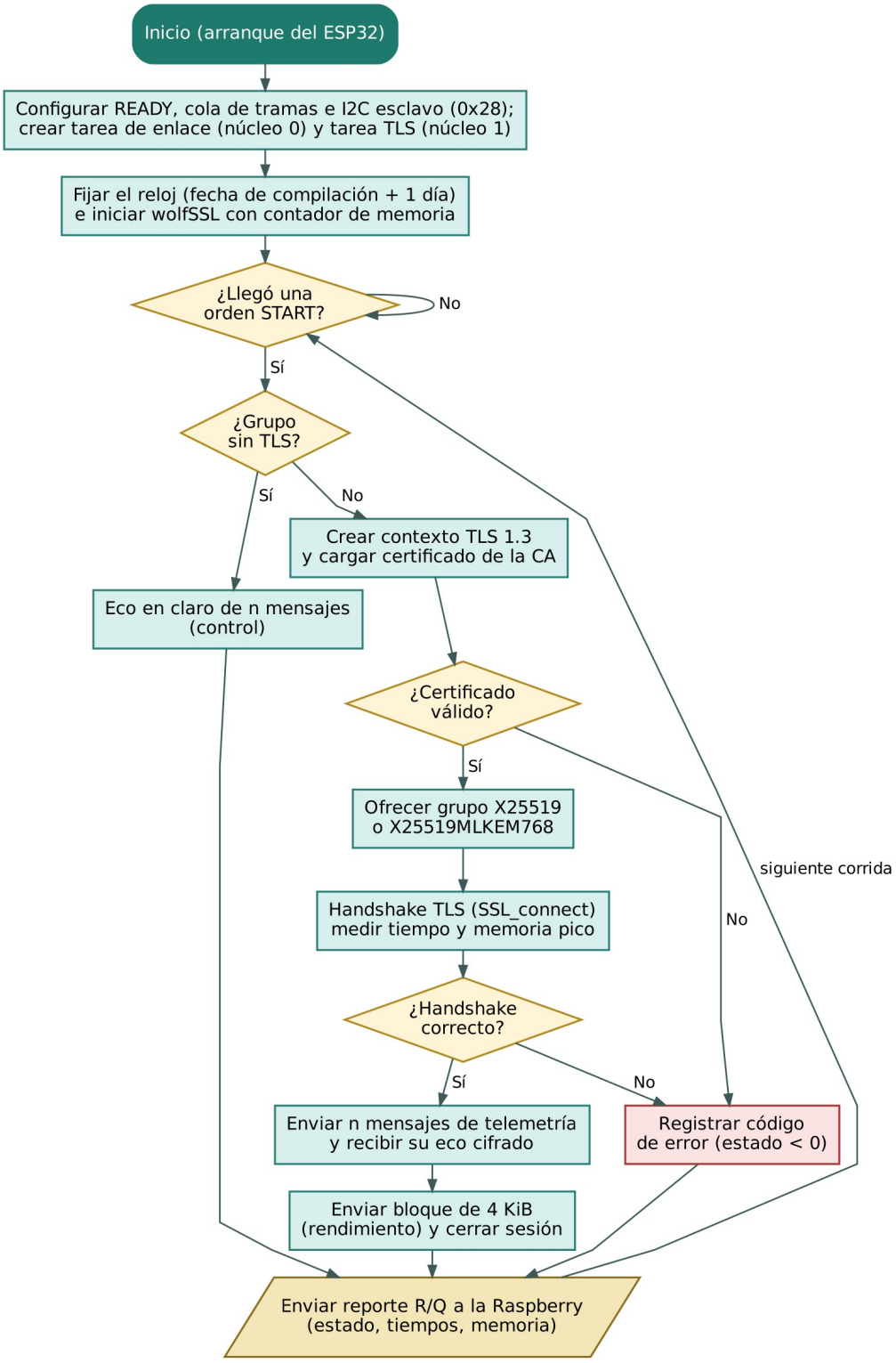
Las Figuras 2 a 8 presentan los diagramas de flujo descritos, uno por página.

Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

Figura 2

Diagrama de flujo del firmware del ESP32 (main.c y cliente_tls.c)



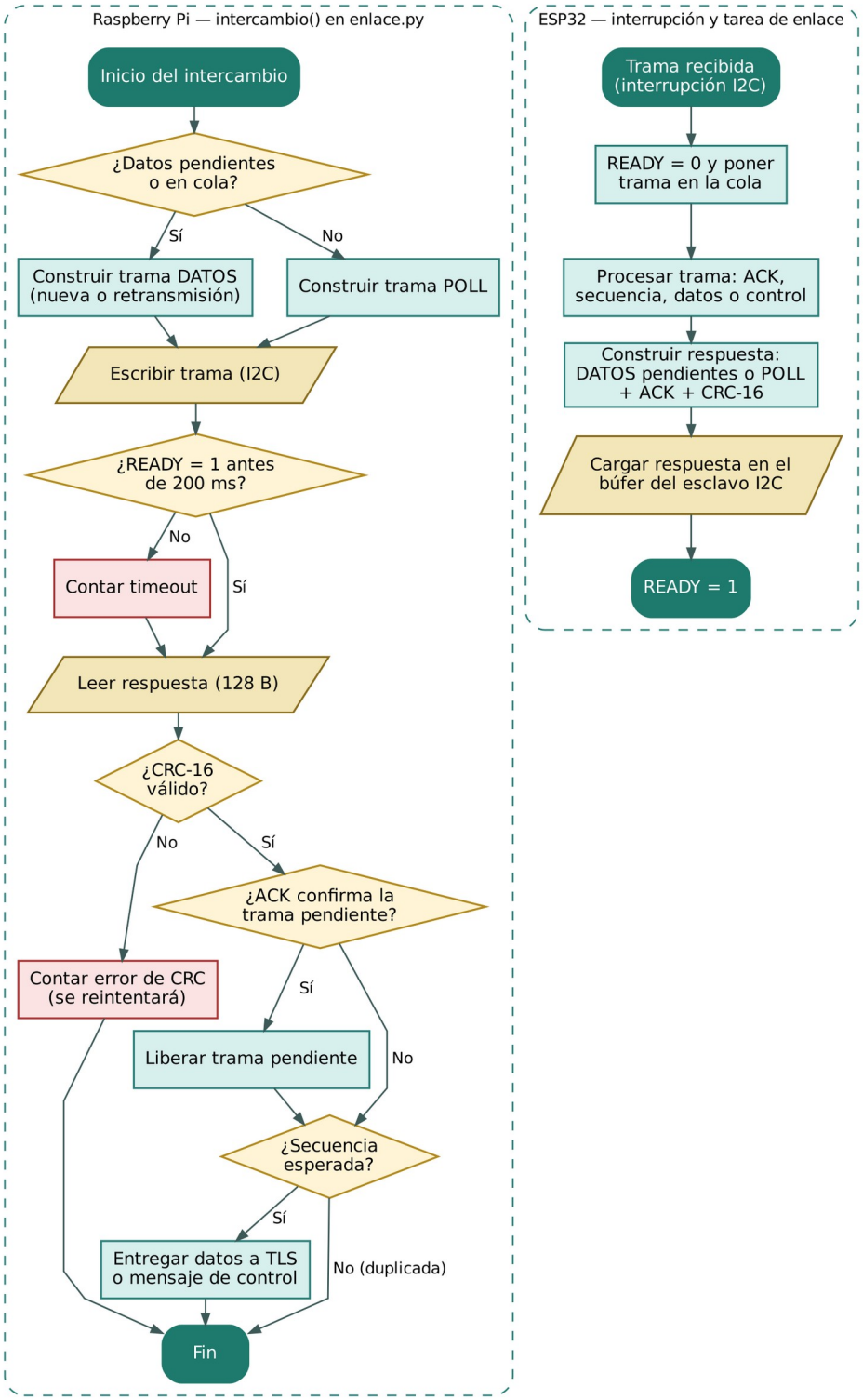
Nota. Elaboración propia. Simbología: terminal (verde oscuro, esquinas redondeadas), proceso (rectángulo), decisión (rombo), entrada o salida de datos (paralelogramo) y tratamiento de errores (rojo).

Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

Figura 3

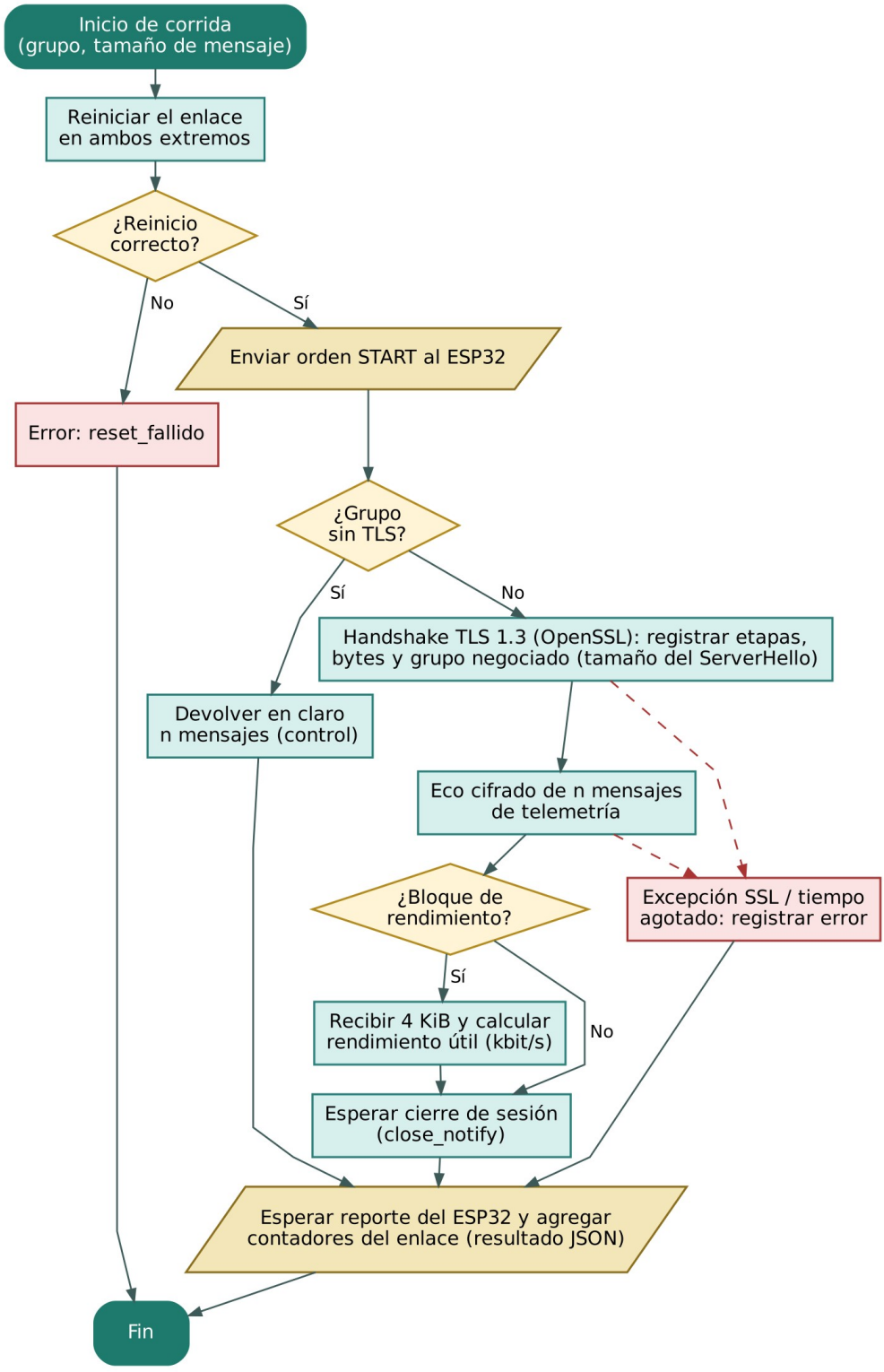
Diagrama de flujo de la capa de enlace en el maestro (enlace.py) y en el esclavo (enlace.c)



Nota. Elaboración propia. Simbología: terminal (verde oscuro, esquinas redondeadas), proceso (rectángulo), decisión (rombo), entrada o salida de datos (paralelogramo) y tratamiento de errores (rojo).

Figura 4

Diagrama de flujo de una corrida en el servidor (servidor_tls.py)



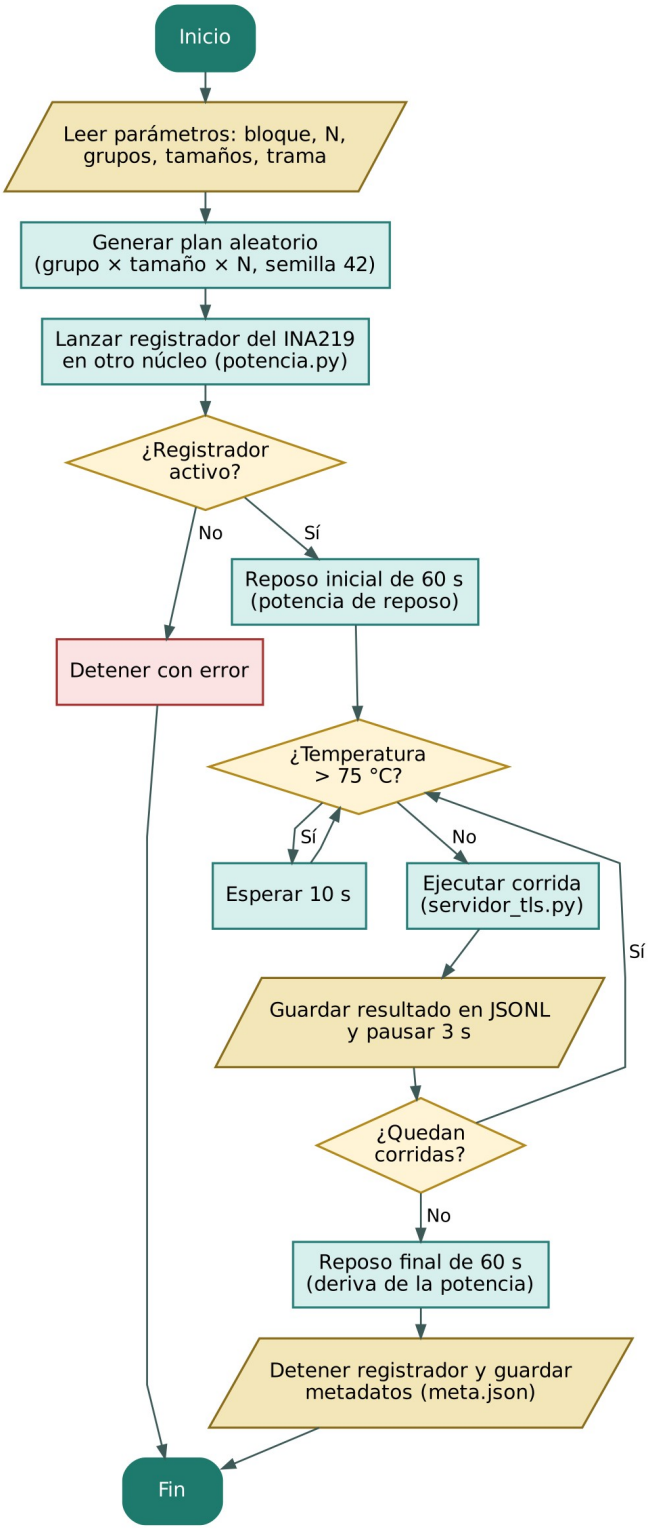
Nota. Elaboración propia. Simbología: terminal (verde oscuro, esquinas redondeadas), proceso (rectángulo), decisión (rombo), entrada o salida de datos (paralelogramo) y tratamiento de errores (rojo).

Nombre: DAVID YELA || KEVIN PEREZ

MATERIA INTEGRADORA DE TELECOMUNICACIONES

Figura 5

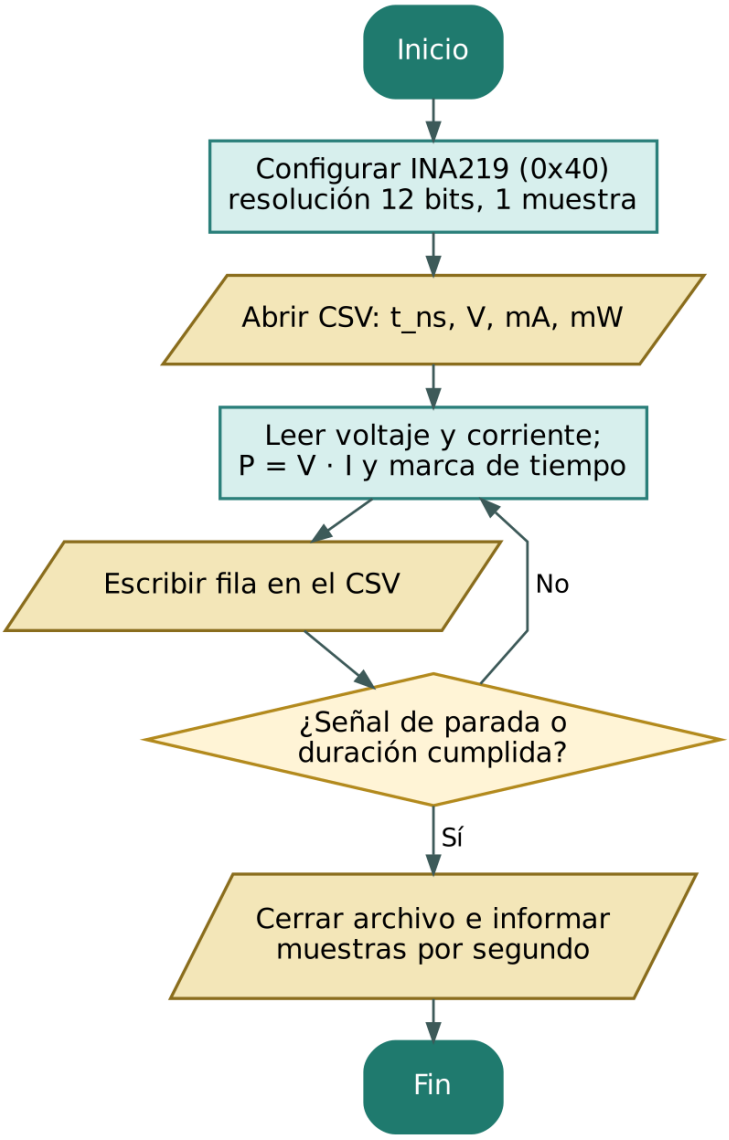
Diagrama de flujo del orquestador de la campaña (campana.py)



Nota. Elaboración propia. Simbología: terminal (verde oscuro, esquinas redondeadas), proceso (rectángulo), decisión (rombo), entrada o salida de datos (paralelogramo) y tratamiento de errores (rojo).

Figura 6

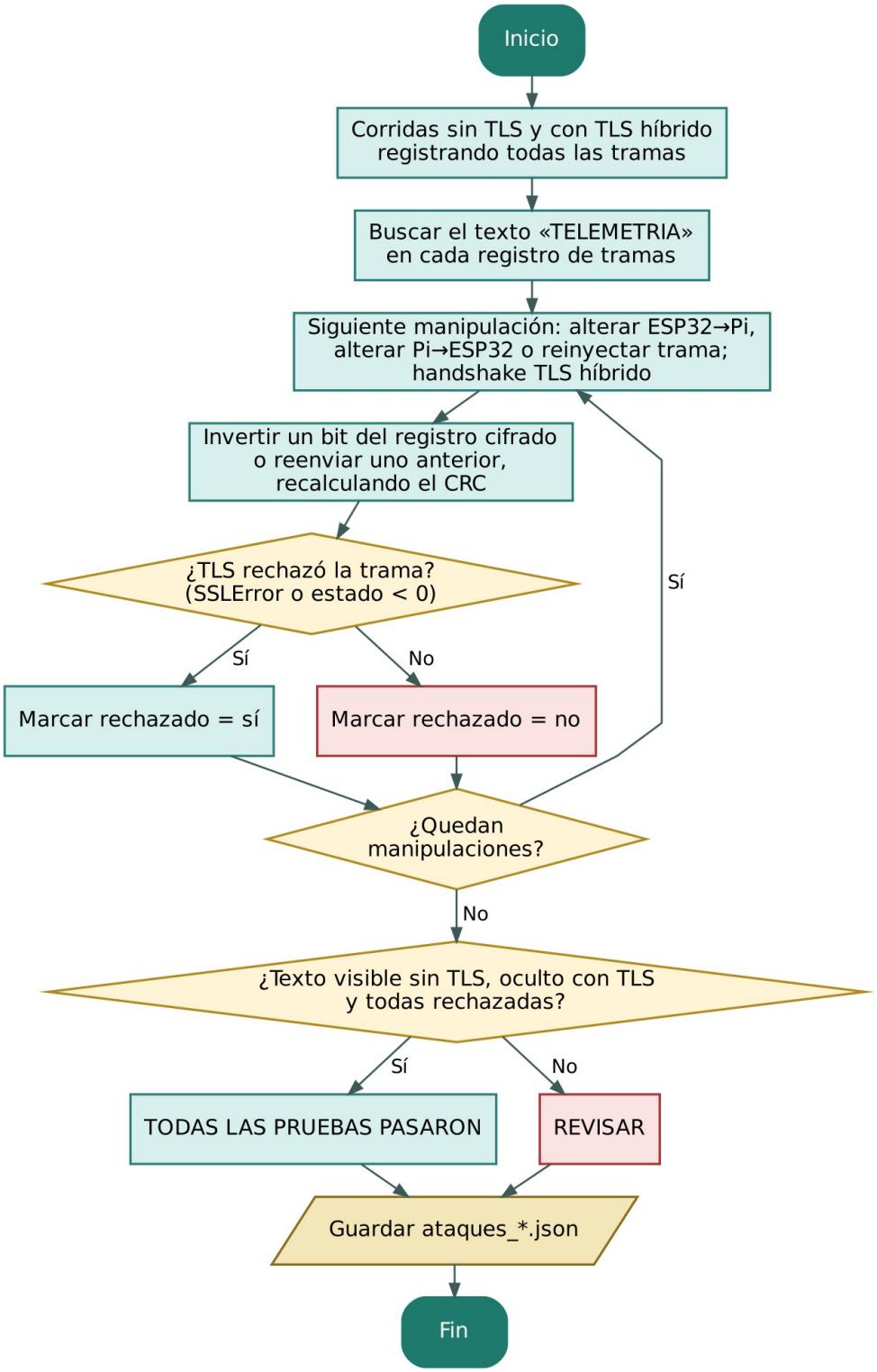
Diagrama de flujo del registrador de potencia (potencia.py)



Nota. Elaboración propia. Simbología: terminal (verde oscuro, esquinas redondeadas), proceso (rectángulo), decisión (rombo), entrada o salida de datos (paralelogramo) y tratamiento de errores (rojo).

Figura 7

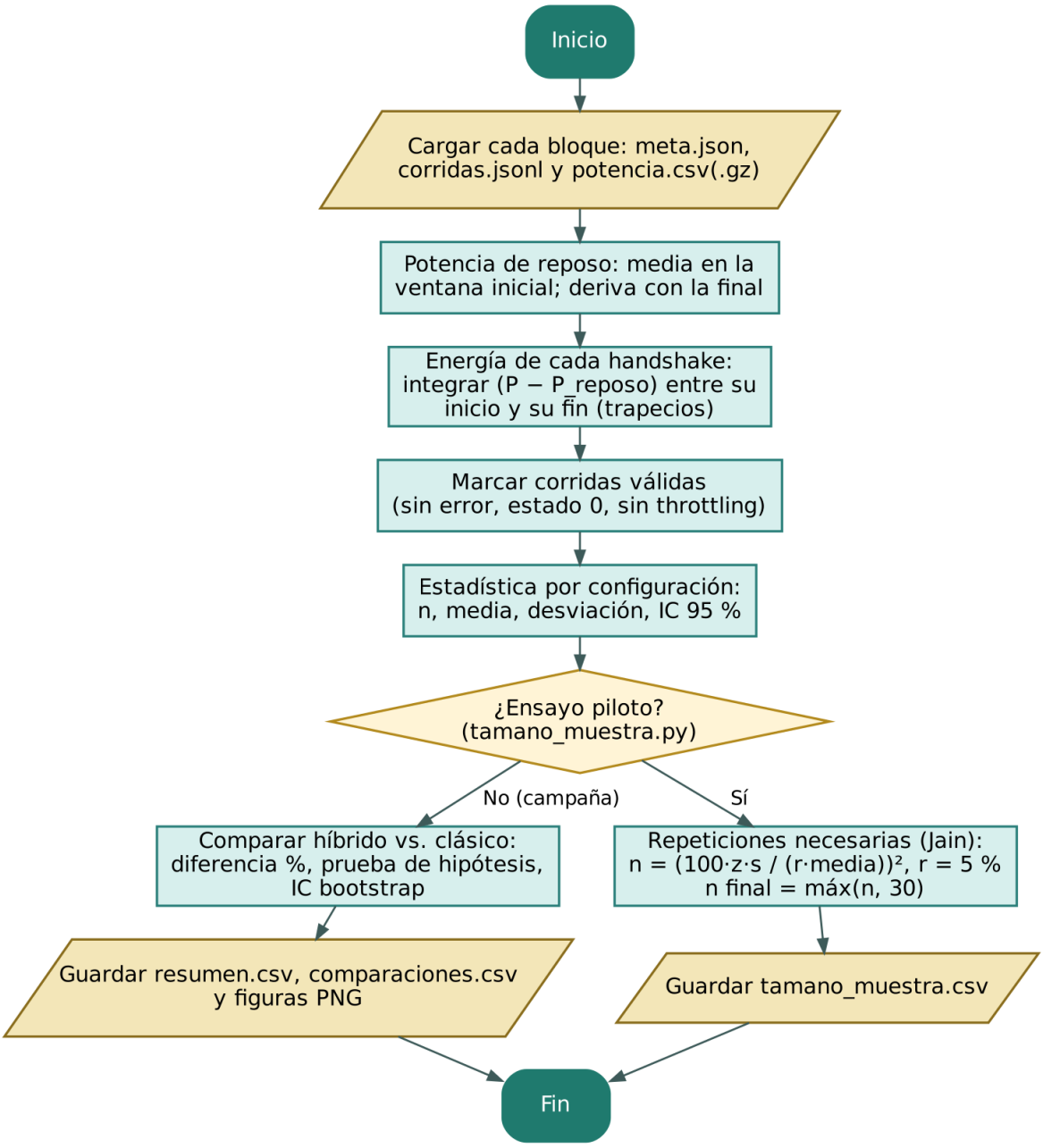
Diagrama de flujo de las pruebas de seguridad (ataque.py)



Nota. Elaboración propia. Simbología: terminal (verde oscuro, esquinas redondeadas), proceso (rectángulo), decisión (rombo), entrada o salida de datos (paralelogramo) y tratamiento de errores (rojo).

Figura 8

Diagrama de flujo del análisis de datos (analisis.py y tamano_muestra.py)



Nota. Elaboración propia. Simbología: terminal (verde oscuro, esquinas redondeadas), proceso (rectángulo), decisión (rombo), entrada o salida de datos (paralelogramo) y tratamiento de errores (rojo).

4) Referencias

Jain, R. (1991). The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling. John Wiley & Sons.